

教研工坊-前端数据与路由设计

教研工坊前端数据与路由设计

1. 文档目的

本文件在前几篇文档基础上，定义：

- 前端路由结构
- 页面对应的数据模型
- 前端 view-model 与 .workshop 运行时的映射关系
- 各页面最关键的数据来源与派生逻辑

目标是让前端设计直接贴着 course-workshop 当前运行时落地，而不是发明一套新的业务状态。

2. 路由总表

2.1 一级路由

```
/
/inbox
/projects
/plans
/knowledge
/exports
/settings
```

2.2 二级路由

```
/projects/:projectId
/projects/:projectId/framing
/projects/:projectId/month
/projects/:projectId/week/:weekId
/projects/:projectId/activities/:activityId
/projects/:projectId/review
/projects/:projectId/export

/plans/:planId
/knowledge/examples
/knowledge/library
/exports/:projectId/:target
```

3. 路由与运行时映射

/projects

来源：

- .workshop/projects/*/status.json

展示内容：

- 项目列表
- phase
- HIL
- pipeline
- 最近更新时间
- 缺失项

/projects/:projectId

来源：

- .workshop/projects/{projectId}/status.json
- .workshop/projects/{projectId}/config.yaml

- 项目目录中的 artifact 存在情况

用途：

- Project Workspace 总控页

/projects/:projectId/framing

来源：

- theme-analysis.md
- theme-narrative.md
- theme-network.md
- status.json.hil

/projects/:projectId/month

来源：

- month-plan.md
- theme-network.md
- status.json.skills

/projects/:projectId/week/:weekId

来源：

- week-plan.md
- 关联活动稿索引
- status.json.plan_refs

/projects/:projectId/activities/:activityId

来源：

- 单个活动稿文件
- 如：
 - lesson-plan.md
 - activities/*.md
 - thematic activity files

/projects/:projectId/review

来源:

- quality-report.md
- review-comments.md
- resource-plan.md
- resource-check-report.md

/projects/:projectId/export

来源:

- courses/{projectId}/
- .workshop/exports/{projectId}/
- status.json.target

/plans

来源:

- .workshop/plans/*/status.json

/plans/:planId

来源:

- .workshop/plans/{planId}/
- status.json
- semester-plan.md / month-plan.md / week-plan.md

/knowledge

来源:

- .workshop/kb/
- workshop-kb/references/client-examples/

4. 核心前端数据模型

建议前端只做一层轻量 view-model，不重新发明领域模型。

4.1 ProjectSummary

```
type ProjectSummary = {  
  id: string  
  theme?: string  
  pipeline?: string  
  phase: 'planning' | 'designing' | 'reviewing' | 'approved' |  
    'shipped'  
  hil: {  
    checkpoint: 'project-framing' | 'design-scaffold' |  
      'deliverable-draft' | 'approval-gate'  
    status: 'not_started' | 'awaiting_review' | 'changes_requested'  
      | 'approved'  
  }  
  planRefs: {  
    semester?: string | null  
    month?: string | null  
    week?: string | null  
  }  
  skillsDone: number  
  skillsTotal: number  
  updatedAt?: string  
}
```

用途：

- Inbox 列表
- Projects 列表
- 项目摘要卡

4.2 ProjectWorkspace

```
type ProjectWorkspace = {  
  summary: ProjectSummary  
  deliverables: DeliverableCard[]  
}
```

```
    participants: Participant[]
    timeline: TimelineEvent[]
    nextActions: CopilotAction[]
  }
```

用途：

- Project Workspace 总控页

4.3 DeliverableCard

```
type DeliverableCard = {
  id: 'framing' | 'month' | 'week' | 'activities' | 'review' |
    'export'
  title: string
  status: 'empty' | 'in_progress' | 'ready' | 'blocked'
  artifacts: string[]
  missing: string[]
  cta: {
    label: string
    route: string
  }
}
```

用途：

- 用工作模块卡替代文件树

4.4 HILState

```
type HILState = {
  checkpoint: 'project-framing' | 'design-scaffold' | 'deliverable-
    draft' | 'approval-gate'
  status: 'not_started' | 'awaiting_review' | 'changes_requested' |
    'approved'
  requestedAt?: string | null
  approvedAt?: string | null
  approvedBy?: string | null
}
```

```
    notes?: string
}
```

用途：

- PhaseRail
- GateModal
- Inbox 审批卡

4.5 PlanSummary

```
type PlanSummary = {
  id: string
  type: 'planning'
  level: 'semester' | 'month' | 'week'
  phase: string
  linkedProjects: string[]
  updatedAt?: string
}
```

4.6 ActivityDocument

```
type ActivityDocument = {
  id: string
  activityType: 'teaching' | 'region' | 'outdoor' | 'life-routine' |
    'home-school'
  title: string
  week?: string
  subtheme?: string
  status: 'draft' | 'reviewing' | 'approved'
  sourceFile: string
  sections: ActivitySection[]
}
```

用途：

- Activity Editor

4.7 CopilotContext

```
type CopilotContext = {  
  scope: 'inbox' | 'project' | 'framing' | 'month' | 'week' |  
        'activity' | 'review' | 'export'  
  projectId?: string  
  artifactId?: string  
  phase?: string  
  hil?: HILState  
  missing?: string[]  
  suggestedActions: CopilotAction[]  
}
```

用途：

- 右侧 Copilot Panel 的上下文输入

5. 页面级字段映射

5.1 Inbox

依赖：

- 所有 project 的 ProjectSummary

重点计算：

- awaiting_review
- changes_requested
- ready_for_export

例如：

```
awaitingMyReview = projects.filter(p => p.hil.status ===  
  'awaiting_review')  
needsFix = projects.filter(p => p.hil.status ===  
  'changes_requested')  
readyForExport = projects.filter(p => p.phase === 'approved')
```


5.2 Project Workspace

依赖：

- ProjectSummary
- deliverable existence
- linked plans

核心判断：

- framing 是否 ready
- month / week 是否 ready
- review 是否 ready
- export 是否 ready

5.3 Theme Framing

依赖：

- theme-analysis
- theme-narrative
- theme-network
- hil

页面状态：

- 空
- 进行中
- 待确认
- 已通过

5.4 Month Matrix

依赖：

- month-plan
- theme-network
- client examples 可选

建议：

- month-plan 输出应带固定锚点，避免前端猜测结构

5.5 Week Arrangement

依赖：

- week-plan
- activity index

推荐解析结果：

- activity list
- day grouping
- notes
- materials

5.6 Activity Editor

依赖：

- 单个活动稿解析结果

建议：

- 先解析为 section blocks，再进入 UI

5.7 Review

依赖：

- quality-report
- review-comments
- resource-plan
- resource-check-report

关键 view-model：

```
reviewSummary = {  
  hasQuality: boolean  
  hasComments: boolean  
  hasResourcePlan: boolean
```

```
hasResourceCheck: boolean
canEnterApproval: phase === 'reviewing'
}
```

5.8 Export

依赖:

- courses/
- .workshop/exports/
- export manifests

建议 view-model:

```
type ExportTargetView = {
  target: 'word-ready-bundle' | 'pdf-ready-bundle' | 'remote-ready'
  exists: boolean
  manifestPath?: string
  outputFiles: string[]
  ready: boolean
}
```

6. 状态管理建议

6.1 服务端状态

建议由 query 层管理:

- projects
- project detail
- artifacts
- exports
- plans

6.2 本地 UI 状态

建议由 store 管理:

- 当前 tab
- 当前选中的 artifact
- 当前 gate 是否打开
- Copilot 草稿输入
- 局部编辑态

6.3 设计原则

不要把运行时真相复制太多次。

真正的 truth 仍然是：

- `status.json`
- `config.yaml`
- `artifacts`
- `export manifests`

7. 关键建议

7.1 增加 artifact index

如果前端要稳定工作，建议 bridge 提供统一 artifact index，不让每页自己扫目录和猜文件。

7.2 Markdown 要先解析成 view-model

不要把原始 Markdown 直接当 UI 数据结构。

要做：

- `source markdown`
- `parsed artifact view-model`

两层并存。

7.3 路由应围绕工作对象

不要出现：

- `/plugins/xxx`
- `/skills/yyy`

用户应始终停留在：

- 项目
- 计划
- 活动
- 导出

8. 第一阶段最值得先打通的数据链路

如果只做第一阶段，建议先打通这 4 页：

1. Inbox
2. Project Workspace
3. Theme Framing
4. Week Arrangement

因为它们最能体现：

- project-first
- co-pilot
- HIL
- 结构化推进

9. 数据层结论

前端数据设计的关键不在于模型多复杂，而在于：

- 是否贴着 .workshop 运行时
- 是否把文件系统抽象成稳定 view-model
- 是否让 Copilot 总能拿到当前工作上下文

只要这三点成立，前端就能在不破坏现有平台能力的前提下，形成完整的产品体验。